

DOCUMENTAÇÃO TÉCNICA E GUIA DE CONTEXTO DO PROJETO - FAZENDA CAFÉ

Documento gerado para alimentação de IA (Claude / Adapta / Anthropic) e desenvolvedores.
Versão do Sistema: 2026 (Refatoração Modular em Andamento)

1. VISÃO GERAL DO PROJETO

O Fazenda Café (Sistema de Gestão Agrícola para Cafeicultura - Sítio do Morro do Paiol) é uma aplicação web completa de

Principais Objetivos do Sistema:

1. Cadastro e Gestão de Talhões: Controle de área, variedade, altitude, espaçamento e cálculo automático do número estimado de plantas por talhão.
2. Controle de Pulverizações Foliares e Pragas: Calendário por período da lavoura, cadastro de receitas, registro de aplicação.
3. Manejo do Mato: Registro de roçadas, capinas e aplicações de herbicidas por talhão.
4. Análises de Solo e Foliares: Cadastro de laboratórios, parâmetros químicos/físicos, histórico de laudos e geração de relatórios.
5. Adubação e Nutrição: Recomendações técnicas baseadas em análises de solo e registro de adubações realizadas.
6. Controle de Estoque e Insumos: Movimentações (entrada/saída), controle de estoque mínimo, valorização monetária e inventário.
7. Notas Fiscais e Documentos: Cadastro de notas de compra com upload de PDF e vínculo às entradas de estoque.
8. Devolução de Embalagens Vazias: Gestão de logística reversa de defensivos conforme legislação ambiental.
9. Clima e Previsão do Tempo: Integração com API OpenWeatherMap (geolocalização configurada para Campos Gerais - MG).
10. Emissão de Relatórios em PDF: Utilização do ReportLab para relatórios executivos formatados.

2. ARQUITETURA E STACK TECNOLÓGICA

- Linguagem: Python 3.10+
- Framework Web: Flask (Padrão Application Factory com Blueprints)
- Banco de Dados: PostgreSQL
- Driver de Conexão: psycopg2 com pool de conexões (psycopg2.pool.SimpleConnectionPool)
- Frontend: Jinja2 Templates, HTML5 semântico, CSS customizado, JavaScript modular, FontAwesome / Bootstrap Icons.
- Manipulação de Dados & Relatórios: pandas, reportlab
- APIs Externas: OpenWeatherMap API (dados meteorológicos)
- Autenticação: Gerenciamento de sessões com Flask Session + Werkzeug Security (generate_password_hash, check_password_hash)

3. ESTRUTURA DE DIRETÓRIOS DO PROJETO

```
FazendaCafe/
├── .env                # Variáveis de ambiente (DB, Flask Secret, limites de upload)
├── run.py              # Ponto de entrada da aplicação (executa create_app)
├── logs/              # Logs rotativos da aplicação (fazenda_cafe.log)
├── config/            # Configurações globais e infraestrutura de banco
│   ├── __init__.py    # Classes Config, ProductionConfig, DevelopmentConfig
│   └── database.py    # ConexaoBanco (Pool psycopg2), executar_query, helpers de talhões e PDF
├── database/          # Recursos de banco de dados
│   ├── backups/       # Backups automáticos/manuais do PostgreSQL (.sql, .dump)
│   └── scripts/       # Scripts auxiliares de DDL/DML
├── scripts/           # Utilitários de manutenção e automação
│   ├── criar_admin.py # Script para criar o primeiro usuário admin
│   ├── backup.py      # Rotina de backup do banco de dados
│   ├── restaurar_backup.py # Rotina de restauração do banco
│   └── agendar_backup.bat # Script batch para agendamento no Windows Task Scheduler
├── app/               # Pacote principal da aplicação Flask
│   ├── __init__.py    # Application Factory (create_app), filtros Jinja, registro de Blueprints
│   ├── models.py      # Configuração de Logs (RotatingFileHandler) e Handlers de Erro (403, 404, 500)
│   ├── app.py / app_legacy.py # Monolito original mantido para referência durante a refatoração
│   ├── app_replace.py # Rascunho intermediário de migração
│   └── routes/        # Blueprints de rotas (Controladores HTTP)
│       ├── __init__.py
│       ├── auth_routes.py # Rotas de Login (/login) e Logout (/logout) -> Blueprint 'auth'
│       ├── dashboard_routes.py # Rota raiz (/) com todos os KPIs -> Blueprint 'dashboard'
│       ├── talhoes_routes.py # CRUD de Talhões (/talhoes/*) -> Blueprint 'talhoes'
│       ├── [PENDENTES]: # Rotas a serem criadas/modularizadas nos próximos passos:
│       │   ├── estoque_routes.py
│       │   └── adubacao_routes.py
```

```

■ ■ ■■ pulverizacao_routes.py
■ ■ ■■ analises_routes.py
■ ■ ■■ manejo_mato_routes.py
■ ■ ■■ clima_routes.py
■ ■ ■■ notas_fiscais_routes.py
■ ■ ■■ devolucao_embalagens_routes.py
■
■■■ modules/                # Módulos de Domínio & Regras de Negócio (Camada de Serviço/DAO)
■ ■■ __init__.py
■ ■■ auth.py                # Lógica de login, hash de senha, validação e usuários
■ ■■ login_manager.py      # Decorator @login_required e wrappers de autenticação
■ ■■ talhoes.py            # CRUD de talhões, cálculo de espaçamento/pés e PDF
■ ■■ dashboard.py          # Agregações estatísticas para os gráficos e cards
■ ■■ clima.py              # Consumo da API OpenWeatherMap e ícones de tempo
■ ■■ estoque.py            # Gestão de produtos, entradas, saídas, saldos e relatórios
■ ■■ pulverizacao.py        # Receitas, períodos, aplicações, retornos e pragas
■ ■■ adubacao.py           # Recomendações nutricionais e histórico de adubações
■ ■■ analises.py           # Laboratórios, análises de solo/foliar e parâmetros químicos
■ ■■ manejo_mato.py        # Capinas, roçadas e aplicações de herbicida
■ ■■ notas_fiscais.py      # Notas fiscais de compra, upload PDF e vínculo ao estoque
■ ■■ devolucao_embalagens.py # Comprovantes de devolução de embalagens vazias
■
■■■ static/                  # Arquivos estáticos (CSS, JS, Imagens, Uploads)
■ ■■ uploads/              # PDFs de notas, análises e comprovantes
■ ■■ css/                  # Folhas de estilo da interface
■ ■■ js/                   # Scripts de interatividade e gráficos (ex: Chart.js)
■
■■■ templates/              # Telas Jinja2 organizadas por módulo
■ ■■ base.html             # Layout mestre com Sidebar, Topbar, Flash Messages
■ ■■ index.html            # Página inicial alternativa
■ ■■ dashboard.html        # Dashboard executivo principal
■ ■■ login.html            # Tela de autenticação
■ ■■ novo.html             # Template genérico de cadastro
■ ■■ admin/                # Telas administrativas
■ ■■ adubacao/             # Telas de recomendações e aplicações de adubo
■ ■■ analises/             # Telas de laudos, laboratórios e resultados
■ ■■ errors/               # Telas de erro 403, 404 e 500
■ ■■ estoque/              # Telas de insumos, movimentações e notas fiscais
■ ■■ manejo_mato/          # Telas de controle de mato
■ ■■ pdfs/                 # Listagem e download de relatórios gerados
■ ■■ pulverizacao/         # Telas de receitas, aplicações e alertas
■ ■■ talhoes/              # Telas de listagem, cadastro, edição e detalhes
■ ■■ usuarios/             # Gestão de perfil e usuários

```

4. MODELO DE DADOS & ESQUEMA DO BANCO (POSTGRESQL)

4.1. Tabela: usuarios

- id (SERIAL PRIMARY KEY)
- produtor_id (INTEGER)
- nome (VARCHAR(100) NOT NULL)
- login (VARCHAR(50) UNIQUE NOT NULL)
- senha_hash (VARCHAR(200) NOT NULL)
- tipo (VARCHAR(20) DEFAULT 'user') – Tipos: admin, user, agronomista, produtor
- ativo (BOOLEAN DEFAULT TRUE)
- data_cadastro (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
- ultimo_acesso (TIMESTAMP)

4.2. Tabela: talhoes

- id (SERIAL PRIMARY KEY)
- nome (VARCHAR(100) NOT NULL) – Ex: "Talhão 01 - Sede"
- area_hectares (NUMERIC(10,2) NOT NULL)
- data_plantio (DATE)
- variedade_cafe (VARCHAR(100)) – Ex: "Catuaí Vermelho IAC 144", "Mundo Novo", "Arara"
- altitude_media (NUMERIC(10,2)) – Em metros (ex: 950.00)
- espacamento (VARCHAR(50)) – Formato "Linha x Planta" (Ex: "3.5 x 0.8" ou "3.8 x 0.7")
- observacoes (TEXT)
- produtor_id (INTEGER)
- ativo (BOOLEAN DEFAULT TRUE)
- data_cadastro (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

* Cálculo de Pés de Café:

plantas_por_ha = 10.000 / (espacamento_linha * espacamento_planta)
total_pes = round(plantas_por_ha * area_hectares)

4.3. Tabela: produtos_estoque

- id (SERIAL PRIMARY KEY)
- nome (VARCHAR(150) NOT NULL)
- categoria (VARCHAR(50)) – Fungicida, Inseticida, Herbicida, Fertilizante, Adjuvante, Outros
- unidade (VARCHAR(20)) – L, kg, sc, un
- estoque_minimo (NUMERIC(10,2) DEFAULT 0)
- quantidade_atual (NUMERIC(10,2) DEFAULT 0)
- observacoes (TEXT)
- ativo (BOOLEAN DEFAULT TRUE)

4.4. Tabela: movimentacoes_estoque

- id (SERIAL PRIMARY KEY)
- produto_id (INTEGER REFERENCES produtos_estoque(id))
- tipo (VARCHAR(10)) – 'entrada' ou 'saida'
- quantidade (NUMERIC(10,2) NOT NULL)
- valor_unitario (NUMERIC(10,2))
- data_movimento (DATE DEFAULT CURRENT_DATE)
- nota_fiscal_id (INTEGER)
- talhao_id (INTEGER REFERENCES talhoes(id))
- observacoes (TEXT)
- ativo (BOOLEAN DEFAULT TRUE)

4.5. Tabela: notas_fiscais

- id (SERIAL PRIMARY KEY)
- numero_nota (VARCHAR(50) NOT NULL)
- serie (VARCHAR(20))
- data_emissao (DATE)
- data_recebimento (DATE)
- fornecedor (VARCHAR(150) NOT NULL)
- cnpj_fornecedor (VARCHAR(20))
- valor_total (NUMERIC(12,2))
- arquivo_pdf (VARCHAR(255))
- observacoes (TEXT)
- ativo (BOOLEAN DEFAULT TRUE)
- data_cadastro (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

4.6. Tabela: devolucao_embalagens

- id (SERIAL PRIMARY KEY)
- data_devolucao (DATE NOT NULL)
- local_devolucao (VARCHAR(150))
- quantidade_embalagens (INTEGER NOT NULL)
- nome_responsavel (VARCHAR(100))
- numero_comprovante (VARCHAR(50))
- arquivo_pdf (VARCHAR(255))
- observacoes (TEXT)
- ativo (BOOLEAN DEFAULT TRUE)
- data_cadastro (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

4.7. Tabelas de Pulverização & Pragas

- periodos_lavoura: id, nome, descricao, ativo
- receitas: id, nome, periodo_id, descricao, formula_completa, produtos, observacoes, ativo
- aplicacoes_pulverizacao: id, talhao_id, periodo_id, receita_id, data_aplicacao, data_prevista_retorno, data_retorno_r
- pragas_doencas: id, nome, tipo, nivel_dano, ativo
- ocorrencias_pragas_doencas: id, aplicacao_id, talhao_id, praga_doenca_id, nivel_infestacao, data_ocorrencia, observac

4.8. Tabelas de Análises & Adubação

- laboratorios: id, nome, responsavel, telefone, email, endereco, observacoes, ativo
- tipos_analise: id, nome ('Solo', 'Foliar'), ativo
- parametros_analise: id, tipo_analise_id, nome, unidade, ordem_exibicao, ativo
- analises: id, talhao_id, tipo_analise_id, laboratorio_id, data_coleta, data_resultado, numero_protocolo, responsavel
- resultados_analise: id, analise_id, parametro_id, valor
- recomendacoes_adubacao: id, talhao_id, analise_id, data_recomendacao, responsavel_tecnico, observacoes, ativo
- itens_recomendacao_adubacao: id, recomendacao_id, nutriente, quantidade, unidade, fonte_adubo, produto_id, epoca_apli
- adubacoes: id, talhao_id, recomendacao_id, produto_id, data_aplicacao, quantidade_aplicada, unidade, responsavel, obs

4.9. Tabela: manejos_mato

- id (SERIAL PRIMARY KEY)
- talhao_id (INTEGER REFERENCES talhoes(id))
- data_manejo (DATE NOT NULL)

- tipo_manejo (VARCHAR(50)) – Roçada Mecânica, Roçada Manual, Capina Química, Trincha
- produtos (VARCHAR(200))
- dosagem (VARCHAR(50))
- responsavel (VARCHAR(100))
- observacoes (TEXT)

5. REGRAS DE NEGÓCIO E PADRÕES DO CÓDIGO

5.1. Conexão e Execução de SQL

- Nunca criar conexões avulsas sem fechar.
- Utilizar sempre executar_query(query, params, fetch_one=..., fetch_all=...) importado de config.database.
- Todas as tabelas possuem exclusão lógica via flag ativo = FALSE.

5.2. Padrão de Autenticação e Sessão

- O decorator @login_required valida se session.get('user_id') existe.
- Usuários comuns só acessam funcionalidades gerais; ações críticas exigem session.get('tipo') == 'admin'.

5.3. Tratamento de Formatação nos Templates

No app/__init__.py, existem três filtros customizados:

- format_data: Converte datas para DD/MM/YYYY.
- format_moeda: Converte float para R\$ 1.234,56.
- format_quantidade: Formata números sem casas decimais desnecessárias.

6. STATUS ATUAL DA REFATORAÇÃO (O QUE FALTA FAZER)

Concluído:

1. Infraestrutura central (config/database.py, config/__init__.py, .env).
2. Application Factory (app/__init__.py -> create_app()).
3. Sistema de Logs e Handlers de Erro (app/models.py).
4. Módulos de domínio na pasta app/modules/ (todos os 12 arquivos de regras criados e funcionais).
5. Blueprints: auth_routes.py, dashboard_routes.py, talhoes_routes.py.

Próximas Etapas para o Claude / Desenvolvedor:

1. Criar os Blueprints restantes em app/routes/:
 - estoque_routes.py -> Gerenciar produtos, movimentações, entradas, saídas, relatórios.
 - adubacao_routes.py -> Recomendações e aplicações de adubo.
 - pulverizacao_routes.py -> Receitas, períodos, aplicações, retornos e pragas.
 - analises_routes.py -> Laboratórios, laudos de solo/foliar e cadastro de resultados.
 - manejo_mato_routes.py -> Histórico e novos manejos de mato.
 - clima_routes.py -> Endpoint para atualização/detalhes climáticos.
 - notas_fiscais_routes.py -> Upload de notas, listagem e vínculo com estoque.
 - devolucao_embalagens_routes.py -> Registro de devoluções e upload de comprovantes.
2. Descomentar e Registrar os Blueprints em app/__init__.py.
3. Validar consistência dos formulários HTML em app/templates/.
4. Remover com segurança os arquivos legados (app_legacy.py, app_replace.py, app.py monolítico).

7. PROMPT MESTRE PARA COPIAR E ENVIAR AO CLAUDE NO ADAPTA

Olá Claude! Estou trabalhando no sistema "Fazenda Café", uma aplicação web em Python (Flask) e PostgreSQL para gestão o

Estrutura da aplicação:

- Padrão Application Factory (app/__init__.py cria app via create_app()).
- Ponto de entrada: run.py.
- Blueprints em app/routes/ e regras de negócio/SQL em app/modules/.
- Conexão com banco via config.database.executar_query(query, params, fetch_one=..., fetch_all=...) com pool de conexão.
- Layouts em app/templates/ usando Jinja2 com filtros format_data, format_moeda, format_quantidade.

Status atual:

- A base do banco, os módulos em app/modules/ e os blueprints auth, dashboard e talhoes já estão prontos.
- Estamos criando os próximos Blueprints em app/routes/ e ligando-os ao app/__init__.py.

Por favor, siga rigorosamente essas convenções de nomes, imports e modularidade ao gerar o código para mim.